

Java DSA Syntax & Patterns Cheatsheet

 **Focus: Arrays, Strings, Linked Lists, Stacks, Queues, Hash Maps, Basic Recursion, Backtracking**

1. ARRAYS - Essential Syntax & Patterns

Declaration & Initialization

```
java
```

// 1D Array

```
int[] arr = new int[5];           // [0,0,0,0,0]
int[] arr = {1, 2, 3, 4, 5};      // Direct initialization
int[] arr = new int[]{1, 2, 3};   // Alternative
```

// 2D Array

```
int[][] matrix = new int[3][4];   // 3 rows, 4 cols
int[][] matrix = {{1,2}, {3,4}, {5,6}}; // Direct init
```

// ArrayList (Dynamic Array - USE THIS!)

```
ArrayList<Integer> list = new ArrayList<>();
List<Integer> list = new ArrayList<>(); // Preferred
```

Common Operations

java

```
// Length
arr.length           // Array length
list.size()          // ArrayList size

// Access & Modify
arr[i]               // Get/set element
list.get(i)          // ArrayList get
list.set(i, 10);      // ArrayList set
list.add(10);         // Add to end - O(1)
list.add(0, 10);      // Add at start - O(n)
list.remove(i);       // Remove at index - O(n)
list.remove(Integer.valueOf(10)); // Remove value

// Iterate
for(int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}

for(int num : arr) {   // Enhanced for loop
    System.out.println(num);
}
```

Array Utility Methods

```
import java.util.Arrays;
import java.util.Collections;

Arrays.sort(arr);           // Sort ascending
Arrays.sort(arr, Collections.reverseOrder()); // Descending (Integer[] only!)
Collections.sort(list);     // Sort ArrayList
Collections.reverse(list);   // Reverse ArrayList

Arrays.fill(arr, 0);        // Fill with 0
int[] copy = Arrays.copyOf(arr, arr.length); // Copy array
Arrays.toString(arr);       // Print: [1,2,3]

// Binary Search (array MUST be sorted first!)
int index = Arrays.binarySearch(arr, target);
int index = Collections.binarySearch(list, target);

// Compare
Arrays.equals(arr1, arr2);
```

🔥 PATTERN 1: Two Pointers (Super Important!)

```
java
```

// Pattern A: Pointers from both ends moving inward

// Use: Pair sum, palindrome check, reverse

```
int left = 0, right = arr.length - 1;
```

```
while(left < right) {
```

```
    // Process
```

```
    if(arr[left] + arr[right] == target) {
```

```
        // Found pair!
```

```
        return new int[]{left, right};
```

```
    } else if(arr[left] + arr[right] < target) {
```

```
        left++; // Need bigger sum
```

```
    } else {
```

```
        right--; // Need smaller sum
```

```
    }
```

```
}
```

// Pattern B: Slow and Fast pointer (same direction)

// Use: Remove duplicates, move zeros, partition

```
int slow = 0;
```

```
for(int fast = 0; fast < arr.length; fast++) {
```

```
    if(arr[fast] != 0) { // Example: move non-zeros forward
```

```
        arr[slow] = arr[fast];
```

```
        slow++;
```

```
    }
```

```
}
```

```
// After loop, slow points to position after last valid element
```

PATTERN 2: Sliding Window

```
java
```

// Fixed size window

```
int windowSum = 0;
```

```
int k = 3; // window size
```

// Build first window

```
for(int i = 0; i < k; i++) {
```

```
    windowSum += arr[i];
```

```
}
```

```
int maxSum = windowSum;
```

// Slide the window

```
for(int i = k; i < arr.length; i++) {
```

```
    windowSum = windowSum - arr[i-k] + arr[i]; // Remove left, add right
```

```
    maxSum = Math.max(maxSum, windowSum);
```

```
}
```

// Variable size window (use HashMap for frequency)

```
HashMap<Character, Integer> map = new HashMap<>();
```

```
int left = 0, maxLen = 0;
```

```
for(int right = 0; right < s.length(); right++) {
```

```
    char c = s.charAt(right);
```

```
    map.put(c, map.getOrDefault(c, 0) + 1);
```

// Shrink window if invalid

```
while(map.size() > k) { // Example condition
```

```

    char leftChar = s.charAt(left);
    map.put(leftChar, map.get(leftChar) - 1);
    if(map.get(leftChar) == 0) map.remove(leftChar);
    left++;
}
maxLen = Math.max(maxLen, right - left + 1);
}

```

PATTERN 3: Prefix Sum

java

// Build prefix sum array

```

int[] prefix = new int[arr.length];
prefix[0] = arr[0];
for(int i = 1; i < arr.length; i++) {
    prefix[i] = prefix[i-1] + arr[i];
}

```

// Get sum from index i to j in O(1)

```

int sum = prefix[j] - (i > 0 ? prefix[i-1] : 0);

```

Common Array Tricks

java

// Swap elements

```
int temp = arr[i];
```

```
arr[i] = arr[j];
```

```
arr[j] = temp;
```

// Reverse array in-place

```
int left = 0, right = arr.length - 1;
```

```
while(left < right) {
```

```
    int temp = arr[left];
```

```
    arr[left] = arr[right];
```

```
    arr[right] = temp;
```

```
    left++;
```

```
    right--;
```

```
}
```

// Find max/min

```
int max = Integer.MIN_VALUE;
```

```
int min = Integer.MAX_VALUE;
```

```
for(int num : arr) {
```

```
    max = Math.max(max, num);
```

```
    min = Math.min(min, num);
```

```
}
```

// Check if sorted

```
boolean isSorted = true;
```

```
for(int i = 1; i < arr.length; i++) {  
    if(arr[i] < arr[i-1]) {  
        isSorted = false;  
        break;  
    }  
}
```

2. STRINGS - Essential Syntax & Patterns

Declaration & Key Concepts

java

```
String s = "hello";
```

```
String s = new String("hello");
```

// IMPORTANT: Strings are IMMUTABLE in Java!

// Any modification creates a NEW string

// For modifications, use StringBuilder (faster!)

```
StringBuilder sb = new StringBuilder();
```

```
StringBuilder sb = new StringBuilder("hello");
```

Common String Methods

java

<code>s.length()</code>	<i>// Length of string</i>
<code>s.charAt(i)</code>	<i>// Get char at index i</i>
<code>s.substring(i, j)</code>	<i>// Get substring [i, j) - j exclusive!</i>
<code>s.substring(i)</code>	<i>// From i to end</i>
<code>s.indexOf('a')</code>	<i>// First occurrence of 'a' (-1 if not found)</i>
<code>s.lastIndexOf('a')</code>	<i>// Last occurrence</i>
<code>s.indexOf("ab")</code>	<i>// Find substring</i>
<code>s.equals(other)</code>	<i>// Compare strings (DON'T USE ==)</i>
<code>s.equalsIgnoreCase(other)</code>	<i>// Ignore case comparison</i>
<code>s.compareTo(other)</code>	<i>// Lexicographic: <0, 0, >0</i>
<code>s.toLowerCase()</code>	<i>// Convert to lowercase</i>
<code>s.toUpperCase()</code>	<i>// Convert to uppercase</i>
<code>s.trim()</code>	<i>// Remove leading/trailing spaces</i>
<code>s.split(" ")</code>	<i>// Split by space -> String[]</i>
<code>s.split("\\s+")</code>	<i>// Split by any whitespace</i>
<code>String.join(" ", arr)</code>	<i>// Join array with delimiter</i>
<code>s.contains("abc")</code>	<i>// Check if contains substring</i>
<code>s.startsWith("abc")</code>	<i>// Check prefix</i>
<code>s.endsWith("abc")</code>	<i>// Check suffix</i>

```
s.replace('a', 'b')    // Replace all 'a' with 'b'
s.replaceAll("\\s", "") // Replace using regex

s.toCharArray()       // Convert to char[]
```

StringBuilder Methods (IMPORTANT for modifications!)

java

```
StringBuilder sb = new StringBuilder();

sb.append("hello")    // Add to end
sb.append('a')        // Add char
sb.insert(0, "start")  // Insert at position
sb.delete(0, 5)        // Delete from index 0 to 4
sb.deleteCharAt(0)     // Delete char at index 0
sb.reverse()          // Reverse the string
sb.setCharAt(0, 'H')   // Set char at index
sb.charAt(i)          // Get char at index
sb.length()           // Length

sb.toString()         // Convert back to String
```

PATTERN 1: Character Frequency Count

java

// Method 1: Using int array (only for a-z or A-Z)

```
int[] freq = new int[26];  
for(char c : s.toCharArray()) {  
    freq[c - 'a']++; // For lowercase  
    // freq[c - 'A']++; // For uppercase  
}
```

// Method 2: Using HashMap (for any characters)

```
HashMap<Character, Integer> map = new HashMap<>();  
for(char c : s.toCharArray()) {  
    map.put(c, map.getOrDefault(c, 0) + 1);  
}
```

PATTERN 2: Two Pointers for Strings

java

// Check if palindrome

```
boolean isPalindrome(String s) {  
    int left = 0, right = s.length() - 1;  
    while(left < right) {  
        if(s.charAt(left) != s.charAt(right)) {  
            return false;  
        }  
        left++;  
        right--;  
    }  
    return true;  
}
```

// Reverse a string

```
String reverse(String s) {  
    char[] arr = s.toCharArray();  
    int left = 0, right = arr.length - 1;  
    while(left < right) {  
        char temp = arr[left];  
        arr[left] = arr[right];  
        arr[right] = temp;  
        left++;  
        right--;  
    }  
}
```

```
return new String(arr);  
}
```

PATTERN 3: Anagram Detection

java

// Method 1: Sort and compare

```
boolean isAnagram(String s, String t) {  
    if(s.length() != t.length()) return false;  
  
    char[] arr1 = s.toCharArray();  
    char[] arr2 = t.toCharArray();  
    Arrays.sort(arr1);  
    Arrays.sort(arr2);  
  
    return Arrays.equals(arr1, arr2);  
}
```

// Method 2: Frequency count (faster!)

```
boolean isAnagram(String s, String t) {  
    if(s.length() != t.length()) return false;  
  
    int[] count = new int[26];  
    for(int i = 0; i < s.length(); i++) {  
        count[s.charAt(i) - 'a']++;  
        count[t.charAt(i) - 'a']--;  
    }  
  
    for(int c : count) {  
        if(c != 0) return false;  
    }  
}
```

```
return true;
```

```
}
```

Common String Tricks

```
java
```

// Convert String to int

```
int num = Integer.parseInt("123");
```

// Convert int to String

```
String s = String.valueOf(123);
```

```
String s = Integer.toString(123);
```

```
String s = "" + 123; // Quick trick
```

// Convert char to int (digit)

```
char c = '5';
```

```
int digit = c - '0'; // = 5
```

// Check if char is letter/digit

```
Character.isLetter(c);
```

```
Character.isDigit(c);
```

```
Character.isLetterOrDigit(c);
```

```
Character.isWhitespace(c);
```

// Convert case

```
Character.toLowerCase(c);
```

```
Character.toUpperCase(c);
```

3. LINKED LISTS - Essential Syntax & Patterns

Node Definition (MEMORIZE THIS!)

java

```
class ListNode {  
    int val;  
    ListNode next;  
  
    ListNode(int val) {  
        this.val = val;  
        this.next = null;  
    }  
  
    ListNode(int val, ListNode next) {  
        this.val = val;  
        this.next = next;  
    }  
}
```

Basic Operations

java

// Create a linked list: 1 -> 2 -> 3

```
ListNode head = new ListNode(1);  
head.next = new ListNode(2);  
head.next.next = new ListNode(3);
```

// Traverse

```
ListNode current = head;  
while(current != null) {  
    System.out.println(current.val);  
    current = current.next;  
}
```

// Insert at beginning

```
ListNode newHead = new ListNode(0);  
newHead.next = head;  
head = newHead;
```

// Insert after a node

```
void insertAfter(ListNode node, int val) {  
    ListNode newNode = new ListNode(val);  
    newNode.next = node.next;  
    node.next = newNode;  
}
```

// Delete node after given node

```
void deleteAfter(ListNode node) {  
    if(node != null && node.next != null) {  
        node.next = node.next.next;  
    }  
}
```

PATTERN 1: Dummy Head (Simplifies edge cases!)

```
java  
  
// Use when you might modify the head  
ListNode dummy = new ListNode(0);  
dummy.next = head;  
  
// Do operations...  
ListNode current = dummy.next;  
  
// Return new head  
return dummy.next;
```

PATTERN 2: Fast and Slow Pointers (Floyd's Cycle)

```
java
```

// Find middle of linked list

```
ListNode findMiddle(ListNode head) {  
    ListNode slow = head, fast = head;  
    while(fast != null && fast.next != null) {  
        slow = slow.next;    // Move 1 step  
        fast = fast.next.next; // Move 2 steps  
    }  
    return slow; // Middle node  
}
```

// Detect cycle in linked list

```
boolean hasCycle(ListNode head) {  
    ListNode slow = head, fast = head;  
    while(fast != null && fast.next != null) {  
        slow = slow.next;  
        fast = fast.next.next;  
        if(slow == fast) return true; // Cycle found!  
    }  
    return false;  
}
```

// Find cycle start (if cycle exists)

```
ListNode detectCycle(ListNode head) {  
    ListNode slow = head, fast = head;
```

```
// First, detect if cycle exists
while(fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
    if(slow == fast) break;
}

if(fast == null || fast.next == null) return null; // No cycle

// Reset slow to head, move both at same speed
slow = head;
while(slow != fast) {
    slow = slow.next;
    fast = fast.next;
}
return slow; // Cycle start
}
```

PATTERN 3: Reverse Linked List (SUPER IMPORTANT!)

java

// Iterative reversal (MASTER THIS!)

```
ListNode reverse(ListNode head) {  
    ListNode prev = null;  
    ListNode current = head;  
  
    while(current != null) {  
        ListNode nextTemp = current.next; // Save next  
        current.next = prev; // Reverse link  
        prev = current; // Move prev forward  
        current = nextTemp; // Move current forward  
    }  
  
    return prev; // New head  
}
```

// Recursive reversal

```
ListNode reverse(ListNode head) {  
    if(head == null || head.next == null) {  
        return head;  
    }  
  
    ListNode newHead = reverse(head.next);  
    head.next.next = head; // Reverse the link  
    head.next = null; // Break old link  
}
```

```
return newHead;  
}
```

PATTERN 4: Merge Two Sorted Lists

```
java
```

```
ListNode mergeTwoLists(ListNode l1, ListNode l2) {  
    ListNode dummy = new ListNode(0);  
    ListNode current = dummy;  
  
    while(l1 != null && l2 != null) {  
        if(l1.val < l2.val) {  
            current.next = l1;  
            l1 = l1.next;  
        } else {  
            current.next = l2;  
            l2 = l2.next;  
        }  
        current = current.next;  
    }  
  
    // Attach remaining nodes  
    current.next = (l1 != null) ? l1 : l2;  
  
    return dummy.next;  
}
```

PATTERN 5: Remove Nth Node from End

```
java
```

```
ListNode removeNthFromEnd(ListNode head, int n) {  
    ListNode dummy = new ListNode(0);  
    dummy.next = head;  
  
    ListNode fast = dummy;  
    ListNode slow = dummy;  
  
    // Move fast n+1 steps ahead  
    for(int i = 0; i <= n; i++) {  
        fast = fast.next;  
    }  
  
    // Move both until fast reaches end  
    while(fast != null) {  
        fast = fast.next;  
        slow = slow.next;  
    }  
  
    // slow.next is the node to remove  
    slow.next = slow.next.next;  
  
    return dummy.next;  
}
```

Common Linked List Tricks

java

// Get length

```
int getLength(ListNode head) {  
    int count = 0;  
    while(head != null) {  
        count++;  
        head = head.next;  
    }  
    return count;  
}
```

// Check if linked list is palindrome

```
boolean isPalindrome(ListNode head) {  
    // Find middle  
    ListNode slow = head, fast = head;  
    while(fast != null && fast.next != null) {  
        slow = slow.next;  
        fast = fast.next.next;  
    }
```

// Reverse second half

```
    ListNode secondHalf = reverse(slow);
```

// Compare both halves

```
    ListNode firstHalf = head;  
    while(secondHalf != null) {
```

```
    if(firstHalf.val != secondHalf.val) return false;
    firstHalf = firstHalf.next;
    secondHalf = secondHalf.next;
}
return true;
}
```

4. STACKS - Essential Syntax & Patterns

Stack Declaration

```
java

import java.util.Stack;

Stack<Integer> stack = new Stack<>();

// Better alternative (Deque interface)
Deque<Integer> stack = new ArrayDeque<>(); // Faster!
```

Stack Operations

```
java
```

// Using Stack class

```
stack.push(10);      // Add to top - O(1)
int top = stack.pop(); // Remove and return top - O(1)
int top = stack.peak(); // View top without removing - O(1)
boolean empty = stack.isEmpty(); // Check if empty
int size = stack.size(); // Get size
```

// Using Deque as Stack (recommended!)

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);      // Add to top
int top = stack.pop(); // Remove from top
int top = stack.peak(); // View top
boolean empty = stack.isEmpty();
```

PATTERN 1: Balanced Parentheses/Brackets

java

```

boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();

    for(char c : s.toCharArray()) {
        if(c == '(' || c == '{' || c == '[') {
            stack.push(c);
        } else {
            if(stack.isEmpty()) return false;

            char top = stack.pop();
            if(c == ')' && top != '(') return false;
            if(c == '}' && top != '{') return false;
            if(c == ']' && top != '[') return false;
        }
    }

    return stack.isEmpty();
}

```

// Alternative with HashMap

```

boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    HashMap<Character, Character> map = new HashMap<>();
    map.put(')', '(');
    map.put('}', '{');
}

```

```
map.put(']', '[');

for(char c : s.toCharArray()) {
    if(map.containsKey(c)) { // Closing bracket
        if(stack.isEmpty() || stack.pop() != map.get(c)) {
            return false;
        }
    } else { // Opening bracket
        stack.push(c);
    }
}

return stack.isEmpty();
}
```

PATTERN 2: Next Greater Element

java

```

// Find next greater element for each element
int[] nextGreaterElement(int[] arr) {
    int n = arr.length;
    int[] result = new int[n];
    Stack<Integer> stack = new Stack<>(); // Store indices

    // Traverse from right to left
    for(int i = n - 1; i >= 0; i--) {
        // Pop smaller elements
        while(!stack.isEmpty() && arr[stack.peek()] <= arr[i]) {
            stack.pop();
        }

        result[i] = stack.isEmpty() ? -1 : arr[stack.peek()];
        stack.push(i);
    }

    return result;
}

```

🔥 PATTERN 3: Evaluate Expressions (Postfix/RPN)

```
java
```

// Evaluate Reverse Polish Notation: ["2","1","+","3",""]*

```
int evalRPN(String[] tokens) {  
    Stack<Integer> stack = new Stack<>();  
  
    for(String token : tokens) {  
        if(token.equals("+")) {  
            stack.push(stack.pop() + stack.pop());  
        } else if(token.equals("-")) {  
            int b = stack.pop();  
            int a = stack.pop();  
            stack.push(a - b);  
        } else if(token.equals("*")) {  
            stack.push(stack.pop() * stack.pop());  
        } else if(token.equals("/")) {  
            int b = stack.pop();  
            int a = stack.pop();  
            stack.push(a / b);  
        } else {  
            stack.push(Integer.parseInt(token));  
        }  
    }  
  
    return stack.pop();  
}
```

Stack Implementation using Array

java

```
class MyStack {  
    int[] arr;  
    int top;  
    int capacity;  
  
    MyStack(int size) {  
        arr = new int[size];  
        capacity = size;  
        top = -1;  
    }  
  
    void push(int x) {  
        if(top == capacity - 1) {  
            System.out.println("Stack Overflow");  
            return;  
        }  
        arr[++top] = x;  
    }  
  
    int pop() {  
        if(top == -1) {  
            System.out.println("Stack Underflow");  
            return -1;  
        }  
        return arr[top--];  
    }  
}
```

```
}

int peek() {
    if(top == -1) return -1;
    return arr[top];
}

boolean isEmpty() {
    return top == -1;
}
}
```

5. QUEUES - Essential Syntax & Patterns

Queue Declaration

```
java
```

```
import java.util.Queue;
import java.util.LinkedList;
import java.util.ArrayDeque;

Queue<Integer> queue = new LinkedList<>();
Queue<Integer> queue = new ArrayDeque<>(); // Faster!
```

Queue Operations

```
java

queue.offer(10);    // Add to rear - O(1) (returns false if fails)
queue.add(10);      // Add to rear - O(1) (throws exception if fails)
int front = queue.poll(); // Remove and return front - O(1) (null if empty)
int front = queue.remove(); // Remove front - O(1) (throws exception if empty)
int front = queue.peek(); // View front without removing - O(1)
boolean empty = queue.isEmpty();
int size = queue.size();
```

PATTERN 1: Implement Queue using Two Stacks

```
java
```

```
class MyQueue {
    Stack<Integer> input;
    Stack<Integer> output;

    public MyQueue() {
        input = new Stack<>();
        output = new Stack<>();
    }

    public void push(int x) {
        input.push(x);
    }

    public int pop() {
        peek(); // Ensure output has elements
        return output.pop();
    }

    public int peek() {
        if(output.isEmpty()) {
            while(!input.isEmpty()) {
                output.push(input.pop());
            }
        }
        return output.peek();
    }
}
```

```
}  
  
public boolean empty() {  
    return input.isEmpty() && output.isEmpty();  
}  
}
```

🔥 PATTERN 2: Implement Stack using Two Queues

java

```
class MyStack {
    Queue<Integer> q1;
    Queue<Integer> q2;

    public MyStack() {
        q1 = new LinkedList<>();
        q2 = new LinkedList<>();
    }

    public void push(int x) {
        q2.offer(x);
        while(!q1.isEmpty()) {
            q2.offer(q1.poll());
        }
        Queue<Integer> temp = q1;
        q1 = q2;
        q2 = temp;
    }

    public int pop() {
        return q1.poll();
    }

    public int top() {
        return q1.peek();
    }
}
```

```
}  
  
public boolean empty() {  
    return q1.isEmpty();  
}  
}
```

Circular Queue Implementation

```
java
```

```
class MyCircularQueue {  
    int[] arr;  
    int front, rear, size, capacity;  
  
    public MyCircularQueue(int k) {  
        arr = new int[k];  
        capacity = k;  
        front = 0;  
        rear = -1;  
        size = 0;  
    }  
  
    public boolean enqueue(int value) {  
        if(isFull()) return false;  
        rear = (rear + 1) % capacity;  
        arr[rear] = value;  
        size++;  
        return true;  
    }  
  
    public boolean dequeue() {  
        if(isEmpty()) return false;  
        front = (front + 1) % capacity;  
        size--;  
        return true;  
    }  
}
```

```
}

public int Front() {
    return isEmpty() ? -1 : arr[front];
}

public int Rear() {
    return isEmpty() ? -1 : arr[rear];
}

public boolean isEmpty() {
    return size == 0;
}

public boolean isFull() {
    return size == capacity;
}
}
```

6. HASH MAPS - Essential Syntax & Patterns

HashMap Declaration

java

```
import java.util.HashMap;  
import java.util.HashSet;  
import java.util.Map;  
  
HashMap<String, Integer> map = new HashMap<>();  
Map<String, Integer> map = new HashMap<>(); // Preferred  
  
HashSet<Integer> set = new HashSet<>();  
Set<Integer> set = new HashSet<>(); // Preferred
```

HashMap Operations

java

```
// Add/Update
map.put("key", 10);           // O(1) average
map.put("key", 20);           // Updates if key exists

// Get
int val = map.get("key");      // Returns value or null
int val = map.getDefault("key", 0); // Returns 0 if not found

// Check
boolean exists = map.containsKey("key"); // O(1)
boolean exists = map.containsValue(10); // O(n)

// Remove
int val = map.remove("key");   // Returns removed value

// Size
int size = map.size();
boolean empty = map.isEmpty();

// Clear all
map.clear();
```

Iterate HashMap

java

// Iterate over keys

```
for(String key : map.keySet()) {  
    System.out.println(key + " = " + map.get(key));  
}
```

// Iterate over values

```
for(Integer value : map.values()) {  
    System.out.println(value);  
}
```

// Iterate over entries (Best way!)

```
for(Map.Entry<String, Integer> entry : map.entrySet()) {  
    String key = entry.getKey();  
    Integer value = entry.getValue();  
    System.out.println(key + " = " + value);  
}
```

HashSet Operations

java

```
set.add(10);           // Add element - O(1)
boolean exists = set.contains(10); // Check - O(1)
set.remove(10);        // Remove - O(1)
int size = set.size();
boolean empty = set.isEmpty();

// Iterate
for(int num : set) {
    System.out.println(num);
}
```

PATTERN 1: Two Sum Problem

```
java
```

```
int[] twoSum(int[] nums, int target) {  
    HashMap<Integer, Integer> map = new HashMap<>();  
  
    for(int i = 0; i < nums.length; i++) {  
        int complement = target - nums[i];  
        if(map.containsKey(complement)) {  
            return new int[] {map.get(complement), i};  
        }  
        map.put(nums[i], i);  
    }  
  
    return new int[] {}; // Not found  
}
```

PATTERN 2: Frequency Counter

java

// Count frequency of elements

```
HashMap<Integer, Integer> frequency(int[] arr) {  
    HashMap<Integer, Integer> map = new HashMap<>();  
    for(int num : arr) {  
        map.put(num, map.getDefault(num, 0) + 1);  
    }  
    return map;  
}
```

// Find element with max frequency

```
int maxFrequency(int[] arr) {  
    HashMap<Integer, Integer> map = new HashMap<>();  
    int maxFreq = 0;  
    int result = arr[0];  
  
    for(int num : arr) {  
        int freq = map.getDefault(num, 0) + 1;  
        map.put(num, freq);  
  
        if(freq > maxFreq) {  
            maxFreq = freq;  
            result = num;  
        }  
    }  
}
```

```
    return result;
}
```

PATTERN 3: Group Anagrams

java

```
List<List<String>> groupAnagrams(String[] strs) {
    HashMap<String, List<String>> map = new HashMap<>();

    for(String s : strs) {
        char[] arr = s.toCharArray();
        Arrays.sort(arr);
        String key = new String(arr);

        if(!map.containsKey(key)) {
            map.put(key, new ArrayList<>());
        }
        map.get(key).add(s);
    }

    return new ArrayList<>(map.values());
}
```

PATTERN 4: Subarray Sum Equals K

java

```
int subarraySum(int[] nums, int k) {  
    HashMap<Integer, Integer> map = new HashMap<>();  
    map.put(0, 1); // Important: empty subarray  
  
    int count = 0, sum = 0;  
  
    for(int num : nums) {  
        sum += num;  
  
        if(map.containsKey(sum - k)) {  
            count += map.get(sum - k);  
        }  
  
        map.put(sum, map.getDefault(sum, 0) + 1);  
    }  
  
    return count;  
}
```

Common HashMap Tricks

java

// Get all keys as list

```
List<String> keys = new ArrayList<>(map.keySet());
```

// Get all values as list

```
List<Integer> values = new ArrayList<>(map.values());
```

// Merge two maps

```
map1.putAll(map2);
```

// Compute if absent

```
map.putIfAbsent("key", 10);
```

// Update value

```
map.put("key", map.get("key") + 1);
```

```
map.merge("key", 1, Integer::sum); // Same as above
```

// Remove all entries with value 0

```
map.values().removeIf(v -> v == 0);
```

7. RECURSION - Essential Patterns

Basic Template

```
java

returnType recursiveFunction(parameters) {
    // Base case - MUST have this!
    if(baseCondition) {
        return baseValue;
    }

    // Recursive case
    // 1. Do some work
    // 2. Make recursive call(s)
    // 3. Process result if needed

    return recursiveFunction(modifiedParameters);
}
```

PATTERN 1: Simple Recursion Examples

```
java
```

// Factorial

```
int factorial(int n) {  
    if(n == 0 || n == 1) return 1; // Base case  
    return n * factorial(n - 1);  
}
```

// Fibonacci

```
int fibonacci(int n) {  
    if(n <= 1) return n; // Base case: fib(0)=0, fib(1)=1  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

// Power

```
int power(int x, int n) {  
    if(n == 0) return 1;  
    return x * power(x, n - 1);  
}
```

// Optimized power (divide and conquer)

```
int power(int x, int n) {  
    if(n == 0) return 1;  
    int half = power(x, n / 2);  
    if(n % 2 == 0) {  
        return half * half;  
    } else {
```

```
        return half * half * x;
    }
}

// Sum of digits
int sumDigits(int n) {
    if(n == 0) return 0;
    return (n % 10) + sumDigits(n / 10);
}

// Count digits
int countDigits(int n) {
    if(n == 0) return 0;
    return 1 + countDigits(n / 10);
}
```

PATTERN 2: Array/String Recursion

java

// Print array elements

```
void printArray(int[] arr, int index) {  
    if(index >= arr.length) return; // Base case  
    System.out.println(arr[index]);  
    printArray(arr, index + 1);  
}
```

// Sum of array

```
int arraySum(int[] arr, int index) {  
    if(index >= arr.length) return 0;  
    return arr[index] + arraySum(arr, index + 1);  
}
```

// Find max in array

```
int findMax(int[] arr, int index) {  
    if(index == arr.length - 1) return arr[index];  
    return Math.max(arr[index], findMax(arr, index + 1));  
}
```

// Reverse string

```
String reverse(String s, int index) {  
    if(index < 0) return "";  
    return s.charAt(index) + reverse(s, index - 1);  
}
```

// Check palindrome

```
boolean isPalindrome(String s, int left, int right) {  
    if(left >= right) return true;  
    if(s.charAt(left) != s.charAt(right)) return false;  
    return isPalindrome(s, left + 1, right - 1);  
}
```

PATTERN 3: Multiple Recursive Calls (Tree Recursion)

java

// Print all numbers from 1 to n

```
void printNumbers(int current, int n) {  
    if(current > n) return;  
    System.out.println(current);  
    printNumbers(current + 1, n);  
}
```

// Print all numbers from n to 1

```
void printReverse(int n) {  
    if(n == 0) return;  
    System.out.println(n);  
    printReverse(n - 1);  
}
```

// Generate all binary strings of length n

```
void generateBinary(String current, int n) {  
    if(current.length() == n) {  
        System.out.println(current);  
        return;  
    }  
    generateBinary(current + "0", n);  
    generateBinary(current + "1", n);  
}
```

8. BACKTRACKING - Essential Patterns

Backtracking Template

java

```
void backtrack(parameters) {  
    // Base case - found solution  
    if(isValidSolution) {  
        processSolution();  
        return;  
    }  
  
    // Try all possibilities  
    for(each choice in choices) {  
        // 1. Make choice  
        makeChoice(choice);  
  
        // 2. Explore recursively  
        backtrack(modifiedParameters);  
  
        // 3. Undo choice (backtrack!)  
        undoChoice(choice);  
    }  
}
```

PATTERN 1: Generate Subsets

java

// Generate all subsets of an array

```
List<List<Integer>> subsets(int[] nums) {  
    List<List<Integer>> result = new ArrayList<>();  
    backtrack(nums, 0, new ArrayList<>(), result);  
    return result;  
}  
  
void backtrack(int[] nums, int start, List<Integer> current, List<List<Integer>> result) {  
    // Add current subset to result  
    result.add(new ArrayList<>(current)); // IMPORTANT: Create new list!  
  
    for(int i = start; i < nums.length; i++) {  
        current.add(nums[i]);           // Choose  
        backtrack(nums, i + 1, current, result); // Explore  
        current.remove(current.size() - 1); // Unchoose (backtrack)  
    }  
}
```

PATTERN 2: Generate Permutations

java

// Generate all permutations of an array

```
List<List<Integer>> permute(int[] nums) {  
    List<List<Integer>> result = new ArrayList<>();  
    backtrack(nums, new ArrayList<>(), result);  
    return result;  
}
```

```
void backtrack(int[] nums, List<Integer> current, List<List<Integer>> result) {
```

// Base case - permutation is complete

```
    if(current.size() == nums.length) {  
        result.add(new ArrayList<>(current));  
        return;  
    }
```

```
    for(int num : nums) {  
        if(current.contains(num)) continue; // Skip if already used
```

```
        current.add(num);           // Choose  
        backtrack(nums, current, result); // Explore  
        current.remove(current.size() - 1); // Unchoose
```

```
    }  
}
```

// Better approach using boolean array

```
List<List<Integer>> permute(int[] nums) {
```

```
List<List<Integer>> result = new ArrayList<>();
boolean[] used = new boolean[nums.length];
backtrack(nums, new ArrayList<>(), used, result);
return result;
}

void backtrack(int[] nums, List<Integer> current, boolean[] used, List<List<Integer>> result) {
    if(current.size() == nums.length) {
        result.add(new ArrayList<>(current));
        return;
    }

    for(int i = 0; i < nums.length; i++) {
        if(used[i]) continue;

        current.add(nums[i]);
        used[i] = true;
        backtrack(nums, current, used, result);
        current.remove(current.size() - 1);
        used[i] = false;
    }
}
```

PATTERN 3: Combination Sum

java

// Find all combinations that sum to target (can reuse elements)

```
List<List<Integer>> combinationSum(int[] candidates, int target) {  
    List<List<Integer>> result = new ArrayList<>();  
    Arrays.sort(candidates); // Optional: helps with optimization  
    backtrack(candidates, target, 0, new ArrayList<>(), result);  
    return result;  
}
```

```
void backtrack(int[] candidates, int target, int start, List<Integer> current, List<List<Integer>> result) {  
    if(target == 0) {  
        result.add(new ArrayList<>(current));  
        return;  
    }
```

```
    if(target < 0) return; // Exceeded target
```

```
    for(int i = start; i < candidates.length; i++) {  
        current.add(candidates[i]);  
        backtrack(candidates, target - candidates[i], i, current, result); // i (not i+1) allows reuse  
        current.remove(current.size() - 1);  
    }  
}
```

PATTERN 4: Generate Parentheses

java

// Generate all valid combinations of n pairs of parentheses

```
List<String> generateParenthesis(int n) {  
    List<String> result = new ArrayList<>();  
    backtrack("", 0, 0, n, result);  
    return result;  
}  
  
void backtrack(String current, int open, int close, int max, List<String> result) {  
    if(current.length() == max * 2) {  
        result.add(current);  
        return;  
    }  
  
    if(open < max) {  
        backtrack(current + "(", open + 1, close, max, result);  
    }  
  
    if(close < open) {  
        backtrack(current + ")", open, close + 1, max, result);  
    }  
}
```

PATTERN 5: Word Search in Grid

java

// Search for a word in 2D grid

```
boolean exist(char[][] board, String word) {  
    for(int i = 0; i < board.length; i++) {  
        for(int j = 0; j < board[0].length; j++) {  
            if(backtrack(board, word, i, j, 0)) {  
                return true;  
            }  
        }  
    }  
    return false;  
}
```

```
boolean backtrack(char[][] board, String word, int i, int j, int index) {  
    // Base cases  
    if(index == word.length()) return true; // Found complete word  
    if(i < 0 || i >= board.length || j < 0 || j >= board[0].length) return false;  
    if(board[i][j] != word.charAt(index)) return false;  
  
    // Mark as visited  
    char temp = board[i][j];  
    board[i][j] = '#';  
  
    // Explore all 4 directions  
    boolean found = backtrack(board, word, i+1, j, index+1) ||  
                    backtrack(board, word, i-1, j, index+1) ||
```

```
        backtrack(board, word, i, j+1, index+1) ||
        backtrack(board, word, i, j-1, index+1);

    // Restore (backtrack)
    board[i][j] = temp;

    return found;
}
```



QUICK REFERENCE TIPS

Time Complexity Cheat Sheet

$O(1)$ - Constant: array access, hash get/put
 $O(\log n)$ - Binary search, balanced tree operations
 $O(n)$ - Single loop, traversal
 $O(n \log n)$ - Sorting (merge, quick, heap sort)
 $O(n^2)$ - Nested loops, bubble sort
 $O(2^n)$ - Recursive subsets, fibonacci (without memoization)
 $O(n!)$ - Permutations

Space Complexity

- $O(1)$ - In-place operations (swap, iteration)
- $O(n)$ - Extra array/hash map, recursion depth
- $O(n^2)$ - 2D array

Common Edge Cases to Check

java

// Always test these:

- Empty input: `arr.length == 0`, `s.isEmpty()`
- Single element: `arr.length == 1`
- Two elements: `arr.length == 2`
- All same elements: `[5, 5, 5, 5]`
- Already sorted/reversed
- Duplicates
- Negative numbers
- Zero
- Very large numbers (overflow)
- null input

Input/Output in Java

java

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);

// Read integer
int n = sc.nextInt();

// Read string
String s = sc.next();    // Single word
String line = sc.nextLine(); // Whole line

// Read array
int n = sc.nextInt();
int[] arr = new int[n];
for(int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}

// Print
System.out.println(value);
System.out.print(value); // No newline
System.out.printf("%d %s", num, str); // Formatted

sc.close();
```

Common Mistakes to Avoid

java

// 1. String comparison

```
String s1 = "hello";
```

```
String s2 = "hello";
```

```
if(s1 == s2) { }    // WRONG! Compares reference
```

```
if(s1.equals(s2)) { } // CORRECT!
```

// 2. Integer division

```
int result = 5 / 2;    // result = 2 (not 2.5!)
```

```
double result = 5.0 / 2; // result = 2.5
```

// 3. Array index

```
arr[arr.length];    // WRONG! Index out of bounds
```

```
arr[arr.length - 1]; // CORRECT! Last element
```

// 4. Modifying string directly

```
String s = "hello";
```

```
s[0] = 'H';        // WRONG! Strings are immutable
```

```
char[] arr = s.toCharArray();
```

```
arr[0] = 'H';      // CORRECT!
```

// 5. Integer overflow

```
int max = Integer.MAX_VALUE;
```

```
int result = max + 1; // Overflow! Becomes negative
```

```
long result = (long)max + 1; // CORRECT!
```

FINAL EXAM STRATEGY

Before the Test

1.  Review this cheatsheet
2.  Practice 2-3 problems from each topic
3.  Test your IDE setup
4.  Know how to handle input/output
5.  Sleep well!

During the Test

1. **Read ALL questions first** (2-3 minutes)
2. **Solve easier ones first** - Build confidence!
3. **For each problem:**
 - Understand input/output format
 - Think of edge cases
 - Write pseudocode/approach
 - Code carefully

- Test with examples
- Submit

4. **Time management:**

- Don't spend > 30 -40 min on one problem
- Partial credit is better than nothing
- Move on if stuck

5. **Debugging tips:**

- Print intermediate values
- Check array bounds
- Verify loop conditions
- Test edge cases

Target: 2/4 Problems Correct

- Focus on Arrays, Strings, Hash Maps (easiest to score)
 - Linked Lists, Stacks, Queues (medium difficulty)
 - Recursion/Backtracking (if time permits)
-

Good luck! You've got this! 🎯